C# Day 02 — Questions & Self Study

Giza01Day02 — Review Sheet

Part 1 — Questions & Answers

Q1. What is the shortcut to comment and uncomment a selected block of code in Visual Studio?

- Comment: Ctrl + K, Ctrl + C
- Uncomment: Ctrl + K, Ctrl + U

Q2. Explain the difference between a runtime error and a logical error, with examples.

A runtime error occurs while the program is running and causes the program to stop or throw an exception.

```
int x = 10;
int y = 0;
int result = x / y; // Runtime error: Division by zero
```

A logical error occurs when the program runs successfully, but produces an incorrect result because the logic is wrong.

```
int x = 10;
int y = 20;
int sum = x - y; // Logical error: should be x + y
Console.WriteLine(sum);
```

- Runtime Error: the program fails during execution.
- Logical Error: the program runs, but gives the wrong output.

Q3. Why is it important to follow naming conventions such as PascalCase in C#?

Following naming conventions such as PascalCase in C# is important because it makes the code clear, readable, consistent, and easier to maintain. It also helps developers understand the purpose of classes, methods, and other identifiers quickly.

Q4. Explain the difference between value types and reference types in terms of memory allocation.

Value Types: store the actual value directly. When assigned to another variable, a copy of the value is created. Examples: int, double, bool, char, and struct.

Reference Types: store a reference to an object in memory. When assigned to another variable, both variables can refer to the same object. Examples: class, array, and object.

Q5. What will be the output of the following code? Explain why:

```
int a = 2, b = 7;
Console.WriteLine(a % b);
```

Output: 2

The % operator returns the remainder after division. Here a = 2 and b = 7, so 2 ÷ 7 = 0 with a remainder of 2, because 7 cannot fit into 2 even once. So 2 % 7 = 2.

Note: if the first number is smaller than the second number, the remainder is the first number.

Q6. How does the && (logical AND) operator differ from the & (bitwise AND) operator?

&& (Logical AND): used with Boolean conditions. Returns true only when both conditions are true. Uses short-circuit evaluation — the second condition may not be evaluated if the first is false.

& (Bitwise AND): used to perform an AND operation on the individual bits of integer values. When used with Boolean values, it evaluates both operands without short-circuiting.

Q7. Why is explicit casting required when converting a double to an int?

Explicit casting is required because the double may contain a decimal part, which will be lost when converted to an int.

```
double number = 10.5;
int result = (int)number;
Console.WriteLine(result);
// Output: 10
```

The (int) tells C# that we intentionally want to remove the decimal part. In short: double → int requires explicit casting because data may be lost.

Q8. What exception might occur if the input is invalid, and how can you handle it?

If the user enters an invalid value, such as letters instead of a number, int.Parse() will throw a FormatException. It can be handled using a try-catch block:

```
try
{
    int age = int.Parse(Console.ReadLine());
    Console.WriteLine("Age: " + age);
}
catch (FormatException)
{
    Console.WriteLine("Invalid input. Please enter a valid number.");
}
```

In short: FormatException occurs when the input cannot be converted to the required type.

Q9. Given the code below, what is the value of x after execution? Explain why.

```
int x = 5;
int y = ++x + x++;
```

Answer: x = 7 (and y = 12)

The expression is evaluated left to right:

- ++x → pre-increment: x becomes 6, and the value used is 6.
- x++ → post-increment: the value used is 6, then x becomes 7.

Therefore: y = 6 + 6 = 12, and x = 7.

Q10. What's the difference between compiled and interpreted languages, and where does C# fit?

Compiled Languages: the source code is translated into machine code or an intermediate code before execution. This usually makes execution faster.

Interpreted Languages: the code is translated and executed at runtime, generally without producing a traditional standalone machine-code executable first.

C# is primarily a compiled language. The C# compiler compiles source code into Intermediate Language (IL). The .NET Runtime (CLR) then uses JIT (Just-In-Time) compilation to convert the IL into native machine code during execution:

```
C# Source Code
  ↓
C# Compiler
  ↓
Intermediate Language (IL)
  ↓
CLR + JIT
  ↓
Native Machine Code
  ↓
Execution
```

Q11. Compare implicit, explicit, Convert, and Parse casting.

Method	Meaning	Example	Notes
Implicit	Conversion happens automatically when it is safe.	`double x = 10;`	No casting syntax is required.
Explicit	You manually specify the target type.	`int x = (int)10.5;`	May cause data loss.
Convert	Uses the Convert class to convert between compatible types.	`int x = Convert.ToInt32("10");`	Handles different types and provides several conversion methods.
Parse	Converts a string containing a valid value into a specific type.	`int x = int.Parse("10");`	Mainly used when you know the string contains a valid value; otherwise it can throw an exception.

Q12. What is meant by "C# is managed code"?

When we say C# is managed code, it means that C# code runs under the control of the .NET Runtime (CLR) instead of directly managing the computer's resources. The CLR manages important tasks such as:

- Memory management and Garbage Collection (GC)

- Exception handling
- Type safety
- Security and runtime execution

For example, when objects are created, the CLR manages their memory, and the Garbage Collector automatically removes objects that are no longer needed.

In short: Managed Code = code whose execution and memory management are controlled by the .NET Runtime (CLR).

Q13. How does struct compare to class?

Both struct and class can be used to create custom types in C#, but they behave differently: struct is a Value Type, and class is a Reference Type.

```
struct Point
{
    public int X;
}

class Person
{
    public int Age;
}
```

When you assign a struct to another variable, its value is copied. When you assign a class object to another variable, the reference is copied, so both variables can point to the same object.

- struct → Value Type → copies the value.
- class → Reference Type → copies the reference.

Part 2 — Self Study Topics

1. Naming Conventions

Choosing clear and expressive names for variables makes code easier to understand and maintain, and keeps it organized and consistent.

- Expressive Names — should clearly describe purpose, e.g. FirstName
- camelCase — first word lowercase, following words capitalized (e.g. firstName). Common in JavaScript.
- kebab-case — words separated by hyphens (e.g. first-name). Common in Angular.
- PascalCase — each word capitalized (e.g. FirstName). Common in C#.
- Underscore Naming — an underscore before a name (e.g. _firstName); flagged for further review.

2. Manual Memory Management in C

In C, memory management is handled manually by the programmer, who is responsible for allocating memory when needed and releasing it when no longer required. Careful management prevents memory leaks, which happen when allocated memory is not released after use.

3. Stack Memory

The Stack is a region of memory used during program execution to store data related to method calls and local variables. In C#, value types are generally associated with stack allocation when they are local variables, and the stack has a predictable, structured nature.

The stack follows LIFO (Last In, First Out): the last item added is the first removed. When a method is called, its information is placed on the stack, and it is removed automatically when the method finishes — making allocation and deallocation fast and automatic. However, the exact storage location of a value in .NET depends on context, so it's best not to assume every value type always lives on the stack.

4. Heap Memory

The Heap stores objects created from reference types, such as class instances. Unlike the stack, it is designed to handle objects of varying sizes and lifetimes. For example:

```
Person person = new Person();
```

The variable person holds a reference to the object, while the actual object is stored in the managed heap. The heap is managed by the CLR, which allocates memory for objects and, through the Garbage Collector (GC), automatically reclaims memory from objects that are no longer reachable. The heap has an unknown or variable size because different objects contain different amounts of data.

5. Bitwise Operators

Bitwise operators work directly with the individual bits of a value — low-level operations that manipulate data at the binary level:

- & → Bitwise AND
- | → Bitwise OR
- ^ → Bitwise XOR
- << → Left Shift
- >> → Right Shift

They're useful when working with flags, where individual bits represent different settings or states, and in other low-level or binary-manipulation scenarios.

6. Unchecked

The unchecked context relates to how C# handles numeric overflow — when an arithmetic result falls outside the range a data type can store. The unchecked keyword explicitly tells C# not to perform overflow checking for a block or expression:

```
int x = int.MaxValue;
unchecked
{
    x++;
}
Console.WriteLine(x);
```

Instead of throwing an OverflowException, the value wraps around according to the rules of the integer type — the opposite of a checked context, where overflow results in an OverflowException.

7. Parse and Null

Parse converts a string into a specific data type — especially common for user input, since Console.ReadLine() returns a string:

```
string ageInput = Console.ReadLine();
int age = int.Parse(ageInput);

int age2 = int.Parse("20");
decimal salary = decimal.Parse("5000");
double value = double.Parse("10.5");
```

Parse expects the string to contain a valid value for the target type; an invalid format can throw a FormatException. Its exact relationship with null is flagged as a topic for further study.

8. Convert and Null

The Convert class offers another way to convert between data types. Unlike Parse (mainly string → type), Convert supports conversions between many types:

```
string ageInput = Console.ReadLine();
int age = Convert.ToInt32(ageInput);

string salaryInput = Console.ReadLine();
decimal salary = Convert.ToDecimal(salaryInput);
```

Convert is part of the .NET Base Class Library (BCL) and includes methods such as Convert.ToInt32(), Convert.ToDouble(), Convert.ToDecimal(), Convert.ToString(), and Convert.ToBoolean(). Its specific behavior with null (depending on the method) is flagged as a topic for further study.

9. Convert vs Parse — Performance

Parse is tied directly to the target data type (int.Parse("20")), while Convert uses the general-purpose Convert class (Convert.ToInt32("20")). They differ in behavior, supported conversions, null handling, and performance. For most applications the performance difference matters less than picking the method that fits the situation — the exact performance comparison is flagged as a topic for further investigation.